



AI with Rav

Learn AI the way you actually think.



DAY 7 of 30

MACHINE LEARNING

Decision Trees — the "20 Questions" algorithm

WHAT YOU'LL LEARN TODAY

- › How a decision tree makes decisions
- › How it picks the "best question" to ask
- › Why trees are so easy to explain
- › When trees overfit — and the simple fix



Decision Trees — the "20 Questions" algorithm

In One Line: A Decision Tree asks a series of yes/no questions — like the game "20 Questions" — and follows the answers down to a final decision.

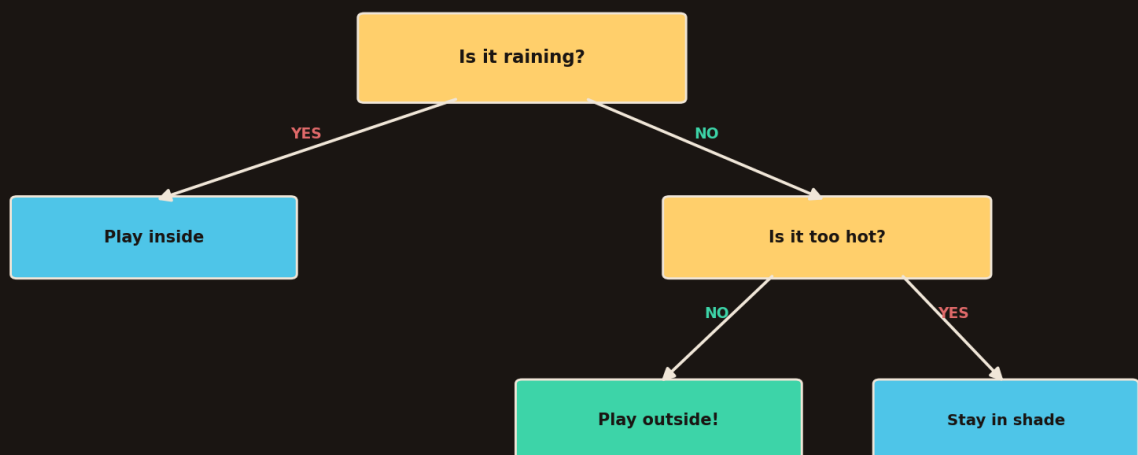
Start here: the 20 Questions game

You've played it: "Is it an animal? Is it bigger than a cat? Does it bark?" Each answer narrows things down until you guess right.

A **Decision Tree** is a machine playing 20 Questions. It asks smart yes/no questions about the data, and each answer sends it down a branch — until it reaches an answer. It's the **most human-like** algorithm, because you can literally read its thinking.

What a decision tree looks like

A Decision Tree = a flowchart of yes/no questions



Each box asks a question; each branch is an answer; the leaves are the final decisions.

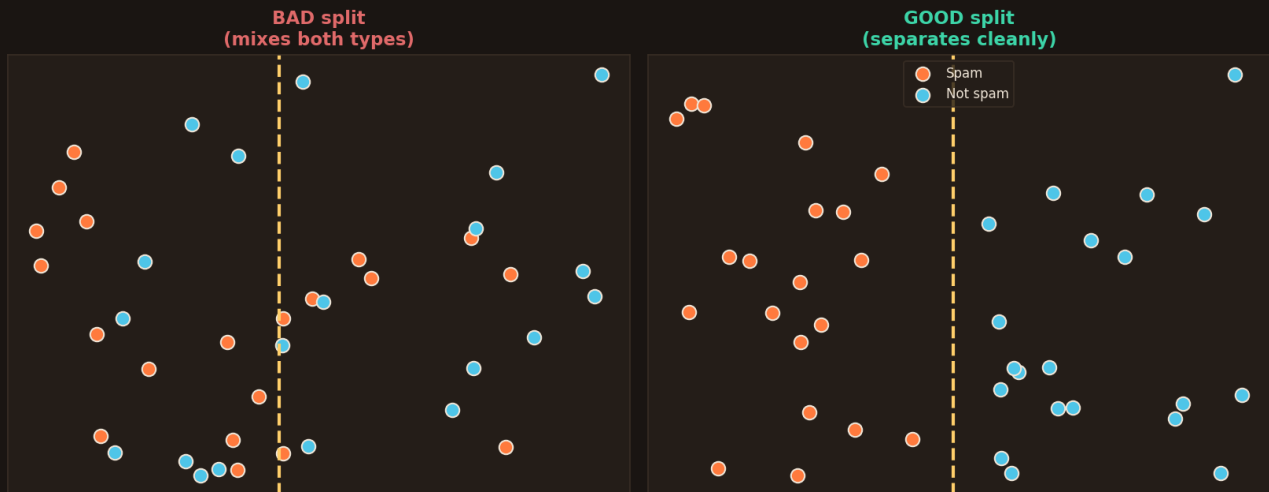
A tree of questions. Each box asks something; each branch is an answer; the leaves are the final decisions.

- Each **box** = a yes/no question about a feature ("Is it raining?").
- Each **branch** = an answer (yes / no).
- Each **leaf** (end) = a final decision ("Play inside").



This works for both jobs: predicting a **category** (spam / not-spam — a classification tree) OR a **number** (house price — a regression tree). Same idea, different leaves.

How does it pick the "best" question?



A good question splits the data cleanly — spam mostly on one side, not-spam on the other. A bad question mixes them up.

The tree doesn't guess questions randomly. At each step it tries every possible question and picks the one that **separates the groups best**:

- A **good** split → one side is mostly "spam", the other mostly "not spam" (clean).
- A **bad** split → both sides are a messy mix (useless).

It measures "messiness" with a score (called *Gini* or *entropy* — just fancy words for "how mixed up is this group?"). It picks the question that reduces the mess the most. That's the entire training process: keep asking the question that tidies the data best.

Why everyone loves decision trees

- **You can SEE the logic** → unlike most ML, you can print the tree and read exactly why it decided something. Huge for trust (banks, doctors, courts).
- **No fancy prep needed** → they handle numbers and categories, don't need scaling.
- **Fast and simple** → easy to train, easy to explain to your boss.

This is the big win: a Decision Tree can explain itself. "Loan rejected because income < 30k AND no credit history." Try getting that from a neural network. This is why trees are everywhere in real business ML.



The catch: overfitting

Trees can grow TOO deep (memorising, not learning)



A tree with too many questions memorises the training data and fails on new data. Fix: limit its depth.

Too shallow = underfit (too simple). Too deep = overfit (memorises). The middle is just right.

If you let a tree ask **too many** questions, it stops learning patterns and starts **memorising** the exact training data — like a student who memorises answers instead of understanding. It looks perfect on training data but fails on new data.

- Too **shallow** → too simple, misses real patterns (underfitting).
- Too **deep** → memorises noise, fails on new data (overfitting).

The fix: limit the tree's depth (e.g. `max_depth=5`) so it can't grow endlessly. We'll go deeper on overfitting in Video 15–16 — for now, just know: a tree left unchecked will cheat by memorising.

Train one — a few lines

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=5) # limit depth = avoid overfit
model.fit(X_train, y_train)

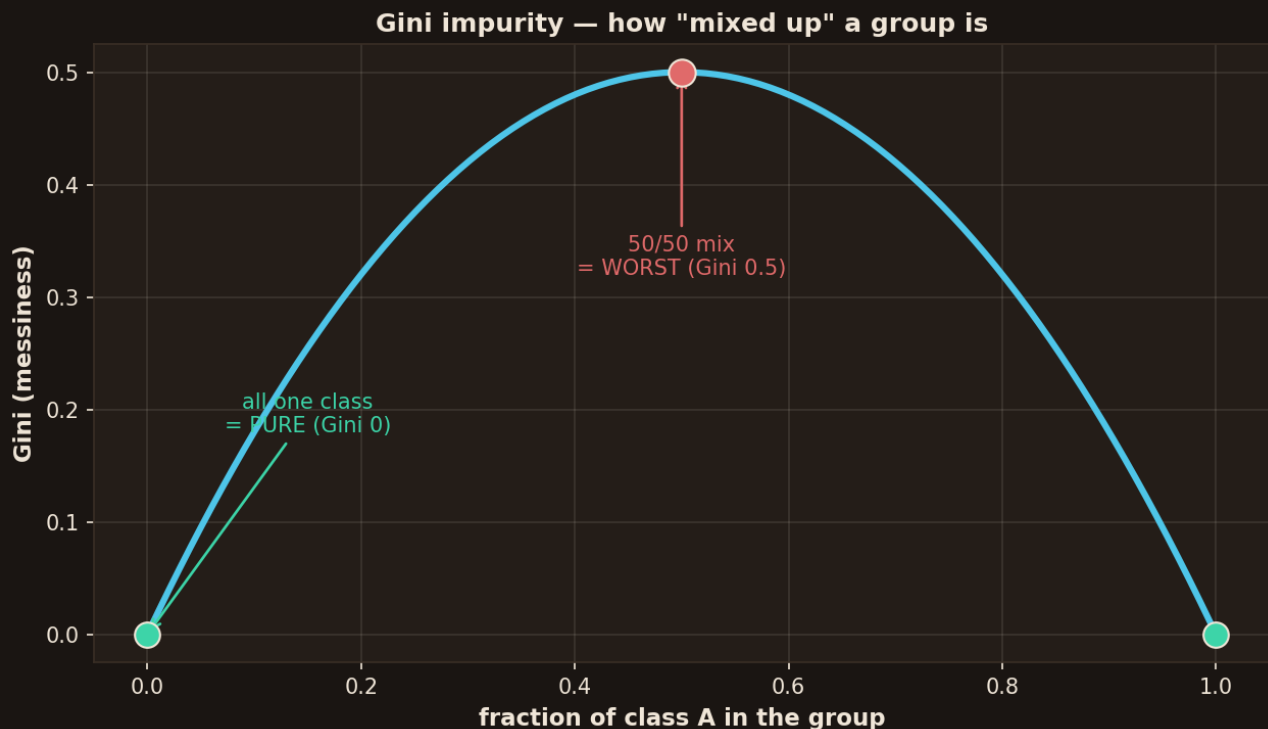
print(model.predict([[income, age, has_credit]]))
# you can even export and READ the whole tree:
from sklearn.tree import export_text
print(export_text(model))
```

`export_text` prints the actual if/else rules the tree learned. No other algorithm lets you read its brain this easily.

■ ■ Under the Hood — the real math (technical, skip if you like)



Non-technical folks: you already have it — jump to the recap. Technical folks: here's exactly how the tree scores its questions.



Gini impurity: 0 when a group is pure (all one class), 0.5 when it's a 50/50 mix. The tree picks splits that push Gini toward 0.

- **Gini impurity** → $\text{Gini} = 1 - \sum (p_i)^2$. For a group that's 50/50 spam/not-spam: $1 - (0.5^2 + 0.5^2) = 0.5$ (worst). All one class: $1 - 1^2 = 0$ (pure). Lower = cleaner.
- **Information gain** → for a candidate question, compute: $\text{Gini}(\text{parent}) - \text{weighted Gini}(\text{children})$. The tree tries every feature and every threshold, and picks the split with the **highest information gain** (the biggest drop in messiness).
- **Entropy** → an alternative to Gini ($-\sum p_i \cdot \log_2(p_i)$) — same idea, from information theory. Gini is faster; results are usually similar.
- **It's greedy** → the tree picks the best split *right now* at each node, never looking ahead. That's why it's fast, but also why it's not always globally optimal (and why a forest of them, Day 8, does better).

When it breaks (what an engineer must know): (1) **Overfitting** — an unrestricted tree grows until every leaf is one sample = memorising. Control with `max_depth`, `min_samples_leaf`, or pruning. (2) **Instability** — change a few rows and the whole tree can restructure; single trees are high-variance (Random Forest fixes this). (3) **Biased to features with many levels** — a column like "user ID" can look like a perfect splitter but generalises to nothing.



Recap — the 20-second version

- A Decision Tree plays **"20 Questions"** — yes/no questions down to a decision.
- It picks each question to **split the data cleanest** (least mixed-up).
- Its superpower: **you can read exactly why** it decided — great for trust.
- Works for categories AND numbers.
- Watch out for **overfitting** (too deep = memorising) — fix with ``max_depth``.

Next up — Video 8: Random Forest. One tree can be wrong or overfit — so what if we ask 100 trees and take a vote? The wisdom of the crowd, applied to ML. See you Day 8.

